Gen AI

15 April 2026 12:38

GEN AI - Generating new content

Initially simple table is easy to handle and we have structured data so easy.
If need to find the given image is cat or dog here data are complicated one so comes deep learning and neural network

Statisctical Method -> neural netwrok -> recural nural network -> generating remaing words.
(Langiage Model)->Transformers

Example Anjela,Thanks for
Contacring me
Reaching out
Finoshing project
Each have differnet probabilty to generate the next words.

We give the datasets or sample to neural network here it does self supervised learning.
Neural netwrok will predict the next words
So many layers in neural network is LLM
Parameters means weights.

Transformer
Google - BERT (text model)
Similarly there will be for image models.
Text - Text Model
BERT
GPT
Text to image
DALL - E
Stable Diffusion
Text to Video
OpenAI Sora

Statistical Probabilty and randomness

GPT - Chatgpt
PaLM 2 - Google
LLaMA- Meta

Open Ai used RLHF to make less toxic
LLM works on database alone that is given.

Neural Networks

15 April 2026 13:35

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.

Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

- **Neurons**: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections**: Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases**: These parameters determine the strength and influence of connections.
- **Propagation Functions**: Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule**: The method that adjusts weights and biases over time to improve accuracy.

1. **Input Computation**: Data is fed into the network.
2. **Output Generation**: Based on the current parameters, the network generates an output.
3. **Iterative Refinement**: The network refines its output by adjusting weights and biases, gradually improving its performance on diverse tasks.

In an adaptive learning environment:

- The neural network is exposed to a simulated scenario or dataset.
- Parameters such as weights and biases are updated in response to new data or conditions.
- With each adjustment, the network's response evolves allowing it to adapt effectively to different tasks or environments.

Layers in Neural Network Architecture

1. **Input Layer:** This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
2. **Hidden Layers:** These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.
3. **Output Layer:** The final layer produces the output of the model. The format of these outputs varies depending on the specific task like classification, regression.

Working of Neural Networks

1. Forward Propagation

When data is input into the network, it passes through the network in the forward direction, from the input layer through the hidden layers to the output layer. This process is known as forward propagation. Here's what happens during this phase:

1. Linear Transformation: Each neuron in a layer receives inputs which are multiplied by the weights associated with the connections. These products are summed together and a bias is added to the sum. This can be represented mathematically as:

$$z = w_1x_1 + w_2x_2 + w_nx_n + b$$

where

- w represents the weights
- x represents the inputs
- b is the bias

2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. The activation function is crucial because it introduces non-linearity into the system, enabling the network to learn more complex patterns. Popular activation functions include ReLU, sigmoid and tanh.

2. Backpropagation

After forward propagation, the network evaluates its performance using a loss function which measures the difference between the actual output and

Neural networks is a computer system inspired by the human brain
Our Brain has

- Neurons
- Neurons connect
- Signals pass between them
- Neural Network has
- Nodes
- Nodes connect

Data passes between them

What is a Neural Network?

A Neural Network is a system that learns patterns from data like the human brain.
Example:

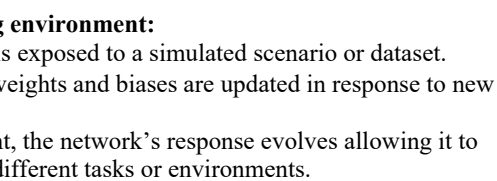
- Input → Student marks
- Output → Pass / Fail

Neural Network Architecture (Main Structure)

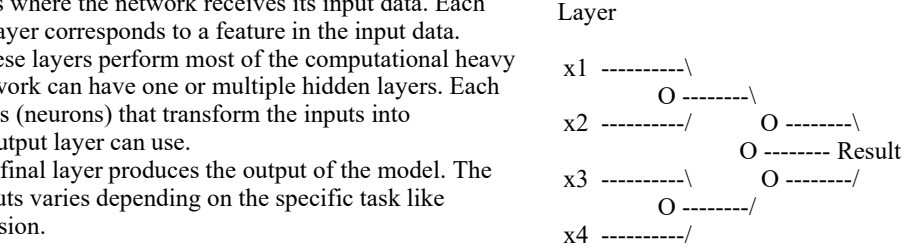
A Neural Network has 3 main layers:

Input Layer → Hidden Layer(s) → Output Layer

Simple Diagram



Full Neural Network Architecture Diagram



Step-by-Step Working

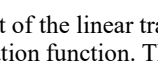
Let's understand with example:
Example
Input:

Study Hours = 5
Sleep Hours = 7

Step 1 — Input Layer

Input Layer receives data:

x1 = 5 (Study Hours)
x2 = 7 (Sleep Hours)
Diagram:



Input layer **does not process** — only passes data.

Step 2 — Hidden Layer

Hidden layer **does the learning**.
Each neuron calculates:

sigmoid and tanh.

2. Backpropagation

After forward propagation, the network evaluates its performance using a loss function which measures the difference between the actual output and the predicted output. The goal of training is to minimize this loss. This is where backpropagation comes into play:

- **Loss Calculation:** The network calculates the loss which provides a measure of error in the predictions. The loss function could vary; common choices are mean squared error for regression tasks or cross-entropy loss for classification.
- **Gradient Calculation:** The network computes the gradients of the loss function with respect to each weight and bias in the network. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.
- **Weight Update:** Once the gradients are calculated, the weights and biases are updated using an optimization algorithm like stochastic gradient descent (SGD). The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.

3. Iteration

This process of forward propagation, loss calculation, backpropagation and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss and the network's predictions become more accurate.

Through these steps, neural networks can adapt their parameters to better approximate the relationships in the data, thereby improving their performance on tasks such as classification, regression or any other predictive modeling.

Example of Email Classification

Let's consider a record of an email dataset:

Email ID	Email Content	Sender	Subject Line	Label
1	"Get free gift cards now!"	spam@example.com	"Exclusive Offer"	1

To classify this email, we will create a feature vector based on the analysis of keywords such as "free" "win" and "offer"

The feature vector of the record can be presented as:

- "free": Present (1)
- "win": Absent (0)
- "offer": Present (1)

How Neurons Process Data in a Neural Network

In a neural network, input data is passed through multiple layers, including one or more hidden layers. Each neuron in these hidden layers performs several operations, transforming the input into a usable output.

1. Input Layer: The input layer contains 3 nodes that indicates the presence of each keyword.

2. Hidden Layer: The input vector is passed through the hidden layer. Each neuron in the hidden layer performs two primary operations: a weighted sum followed by an activation function.

Weights:

- Neuron H1: [0.5,−0.2,0.3]
- Neuron H2: [0.4,0.1,−0.5]

Input Vector: [1,0,1]

Weighted Sum Calculation

- **For H1:** $(1\times0.5)+(0\times-0.2)+(1\times0.3)=0.5+0+0.3=0.8$
- **For H2:** $(1\times0.4)+(0\times0.1)+(1\times-0.5)=0.4+0-0.5=-0.1$

Activation Function

Here we will use ReLu activation function:

- **H1 Output:** ReLU(0.8)= 0.8
- **H2 Output:** ReLu(-0.1) = 0

3. Output Layer: The activated values from the hidden neurons are sent to the output neuron where they are again processed using a weighted sum and an activation function.

- **Output Weights:** [0.7, 0.2]
- **Input from Hidden Layer:** [0.8, 0]
- **Weighted Sum:** $(0.8\times0.7)+(0\times0.2)=0.56+0=0.56$
- **Activation (Sigmoid):** $\sigma(0.56) = \frac{1}{1+e^{-0.56}} \approx 0.636$

4. Final Classification:

- The output value of approximately **0.636** indicates the probability of the

Step 2 — Hidden Layer

Hidden layer **does the learning**.

Each neuron calculates:

Output = (Input × Weight) + Bias

This is called **Weighted Sum**

Example:

x1 = 5
Weight = 2

x2 = 7
Weight = 3

Bias = 1

Output = (5×2) + (7×3) + 1
Output = 10 + 21 + 1 = 32

What is Weight?

Weights tell **importance**

Example:

Study hours important → weight high
Sleep less important → weight low

What is Bias?

Bias helps shift output:

Without bias → 31
With bias → 32
Bias improves learning.

Step 3 — Activation Function

After calculation, we apply
Activation Function

It decides:

- Should neuron activate?
- How strong?

Example:

32 → activation → 0.95

Common activation functions:

- ReLU
- Sigmoid
- Tanh

Example:
ReLU:

f(x) = max(0, x)

Step 4 — Output Layer

Final layer gives result:

Example:

0.95 → Pass
0.20 → Fail

Full Working Flow

Input
↓
Multiply with weights
↓
Add bias
↓
Activation function
↓
Output

Mathematical Flow

y = Activation((x1*w1 + x2*w2 + b))

- **Activation (Sigmoid):** $\sigma(0.56) = \frac{1}{1+e^{-0.56}} \approx 0.636$

4. Final Classification:

- The output value of approximately **0.636** indicates the probability of the email being spam.
- Since this value is greater than 0.5, the neural network classifies the email as spam (1).

Neural Network

Learning of a Neural Network

1. Learning with Supervised Learning

In supervised learning, a neural network learns from labeled input-output pairs provided by a teacher. The network generates outputs based on inputs and by comparing these outputs to the known desired outputs, an error signal is created. The network iteratively adjusts its parameters to minimize errors until it reaches an acceptable performance level.

2. Learning with Unsupervised Learning

Unsupervised learning involves data without labeled output variables. The primary goal is to understand the underlying structure of the input data (X). Unlike supervised learning, there is no instructor to guide the process. Instead, the focus is on modeling data patterns and relationships, with techniques like clustering and association commonly used.

3. Learning with Reinforcement Learning

Reinforcement learning enables a neural network to learn through interaction with its environment. The network receives feedback in the form of rewards or penalties, guiding it to find an optimal policy or strategy that maximizes cumulative rewards over time. This approach is widely used in applications like gaming and decision-making.

when input will be 0
Good follow-up question
Input becomes 0 in many normal situations — it's very common in a Neural Network.
Let me explain clearly
When Input Can Be 0
1. Feature Not Present (Very Common)
Example: Face detection
Features:

Has glasses = 0 (No)
Has beard = 0 (No)
Smiling = 1 (Yes)
Here **0 means feature not present**
So input can be:

[0, 0, 1]
This is very normal.

2. Pixel Value in Images
In a Convolutional Neural Network, images are numbers:
Example:

0 0 255
0 128 255
Here:

- 0 = black
- 255 = white

So **0 is very common** in images.

3. Text Models (LLM)
In a Large Language Model:
Sometimes padding is added:
Example:

Hello world 0 0 0
0 is used to **fill empty space**.

4. Real-Life Example
Student example:

Study hours = 0

Mathematical Flow

y = Activation((x1*w1 + x2*w2 + b))
Where:

- x = input
- w = weight
- b = bias

Multiple Hidden Layers (Deep Learning)
When we use many hidden layers:

Input → Hidden 1 → Hidden 2 → Hidden 3 → Output
This is called
Deep Learning

How Neural Network Learns
Neural network learns using:
Backpropagation
Steps:

1. Make prediction
2. Compare with actual result
3. Calculate error
4. Adjust weights
5. Repeat

Example:

Prediction = 0.8
Actual = 1

Error = 0.2
Adjust weights to reduce error.

Training Loop

Input
↓
Prediction
↓
Error
↓
Adjust weights
↓
Repeat

Neural Network Code Example (Simple)
Using TensorFlow

```
import tensorflow as tf

model = tf.keras.Sequential([
   tf.keras.layers.Dense(10, activation='relu'),
   tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')

model.fit(X, y, epochs=10)
```

Code Explanation
Dense(10)

tf.keras.layers.Dense(10)

- 10 neurons
- Hidden layer

activation='relu'

activation='relu'
Activation function.

Dense(1)
Output layer:

4. Real-Life Example
Student example:

Study hours = 0
Sleep hours = 7
Here input:

[0, 7]
Study hours = 0 → still valid input.

Why Bias Is Important Here
If input = 0:
Without bias:

Output = 0 × weight = 0
Model can't learn.
With bias:

Output = 0 × weight + bias
Output = bias
Now model still works

Simple Summary
Input becomes 0 when:
- Feature not present
- Pixel is black
- Padding in text
- Value actually zero

That's why **bias is important**.

You're asking very strong fundamentals questions — you're basically learning **deep learning properly**
so for each neuron should i give bias and weight
Yes — **each neuron has its own weights and bias** in a Neural Network.
Let me explain clearly
One Neuron Structure
Each neuron calculates:

Output = (x1*w1 + x2*w2 + x3*w3) + bias
So **each neuron has:**
- Multiple **weights** (one per input)
- One **bias**

Example
Suppose you have 2 inputs:

x1 = 5
x2 = 3
Neuron 1:

w1 = 0.2
w2 = 0.8
bias = 1
Output:

(5 × 0.2) + (3 × 0.8) + 1
= 1 + 2.4 + 1
= 4.4

Multiple Neurons (Important)
If you have **3 neurons**, each has **different weights & bias**.
Example:
Neuron 1

w1 = 0.2
w2 = 0.5
bias = 1
Neuron 2

w1 = 0.7
w2 = 0.3
bias = 2
Neuron 3

Dense(1)
Output layer:

tf.keras.layers.Dense(1)
1 output.

Real Life Example
Face detection:

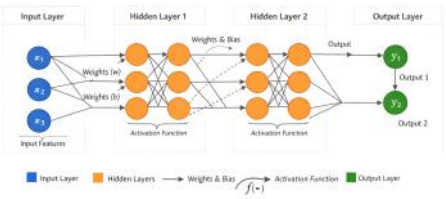
Image
↓
Neural Network
↓
Face detected
Spam detection:

Email
↓
Neural Network
↓
Spam / Not Spam

Final Summary
Neural Network Architecture:

Input Layer
↓
Hidden Layer(s)
↓
Output Layer
Working:

Input → Weight → Bias → Activation → Output



w1 = 0.4
w2 = 0.9
bias = 0.5
Each neuron learns **different patterns**.

Visual Diagram

Inputs Hidden Layer

x1 ——► (Neuron 1)
x2 ——► (Neuron 2)
► (Neuron 3)
Each neuron:
- Has its own weights
- Has its own bias

Why Each Neuron Needs Its Own Bias
Because:
- Each neuron learns **different pattern**
- Each neuron needs **different adjustment**

Example:
- Neuron 1 → detects eyes
- Neuron 2 → detects nose
- Neuron 3 → detects mouth

Each needs different bias.

In CNN Also Same
In a Convolutional Neural Network:
- Each filter has weights
- Each filter has bias

Simple Rule
Each neuron → many weights
Each neuron → one bias

One Line Answer
Yes — every neuron has its own weights and bias.
should i give weight and bias in parameter itself
Short answer: **No — you usually don't manually give weights and bias.**
They are **automatically created and learned during training** in a Neural Network.
Let me explain clearly
How Weights and Bias Are Created
When you define a layer like:

Dense(10)
The framework (like TensorFlow or PyTorch):
- Automatically creates weights
- Automatically creates bias
- Initializes them with random values

Example internally:

weights = random numbers
bias = random numbers
Then during training:
- Model adjusts weights
- Model adjusts bias
- Error decreases
- Model learns

Example in TensorFlow

```
import tensorflow as tf

model = tf.keras.Sequential([
   tf.keras.layers.Dense(10, input_shape=(2,), activation='relu'),
   tf.keras.layers.Dense(1)
])
```

You **did not specify weights or bias**, but they are still created automatically.

What Happens Internally
Suppose:
Input = 2 values
Hidden neurons = 10
Then TensorFlow creates:
- 20 weights (2 × 10)
- 10 bias values

Automatically.

When Do You Give Weights Manually?
Only in **advanced cases**:
Example:

```
Dense(10, use_bias=True)
```

Or manually setting weights:

```
layer.set_weights([weights, bias])
```

But this is **rarely needed**.

Why We Don't Manually Set Weights
Because:
- Model must **learn automatically**
- Manual weights won't work for complex problems
- Learning happens using
  Backpropagation

Training Process

```
Random weights
↓
Prediction
↓
Calculate error
↓
Adjust weights
↓
Repeat
```

Simple Summary
You **don't give weights and bias manually**
Framework automatically:
- Creates them
- Learns them
- Adjusts them

One Line Answer
No — weights and bias are automatically created and learned during training.
how does model adjust wieght and bias without nowing which paramter is important
Excellent question — this is **the core idea of how neural networks learn**
You're asking:
> How does the model adjust weights and bias **without knowing which parameter is important?**

Answer:
It learns using **trial → error → correction** using:
- Backpropagation
- Gradient Descent

Let's break it down step-by-step
Step-by-Step Learning Process
Step 1 — Start With Random Weights
Model starts with **random weights and bias**
Example:

```
Weight = 0.5
Bias = 0.2
```

Model **does not know anything yet**.

Step 2 — Make Prediction
Example:
Input:

Study hours = 5
Actual Output:

Pass = 1
Model Prediction:

0.3
Wrong prediction

Step 3 — Calculate Error
Error = Actual − Predicted

Error = 1 − 0.3
Error = 0.7
This tells model **how wrong it is**.
This is done using
Loss Function

Step 4 — Find Which Weight Caused Error
Model checks:
If I change weight slightly, does error reduce?
Example:
Try increasing weight:

Weight = 0.6
Prediction = 0.5
Error reduced
So model knows:
Increasing weight helps

Step 5 — Adjust Weights
Model updates weight:

Weight = 0.6
This process is called:
Gradient Descent

Full Learning Loop

Random weights
↓
Prediction
↓
Calculate error
↓
Adjust weights
↓
Repeat
This repeats **thousands of times**.

Simple Real-Life Example
Imagine learning to throw a ball
1. Throw randomly
2. Miss target
3. Adjust angle
4. Throw again
5. Improve

Neural network learns the **same way**.

Example With Two Weights
Model:

Output = x1*w1 + x2*w2 + bias
Model tries:

w1 = 0.2
w2 = 0.8
If error high → adjust:

w1 = 0.4
w2 = 0.7
Keep adjusting until **error small**.

Why This Works
Because:
- Small changes tested
- Error checked
- Best direction chosen

This is called:
Backpropagation

Simple Summary
Model doesn't know important parameters initially.
It learns by:
- Trying values
- Checking error
- Adjusting weights
- Repeating

One Line Answer
Model adjusts weights by checking error and gradually improving using backpropagation and gradient descent.

You're now understanding **how AI actually learns**
ok in error fucntion will weight only be increaded or bias wil also be adjusted
Great question
Both weight AND bias are adjusted — not just weight.
In a Neural Network, the model learns by adjusting:
- Weights
- Bias

Both are **trainable parameters**.
Formula Reminder
Neural network calculation:

Output = (x1*w1 + x2*w2) + bias
So if output is wrong, model can fix by:
- Changing **weights**
- Changing **bias**
- Changing **both**

Example
Suppose:

x = 5
w = 2
bias = 1
Output:

5 × 2 + 1 = 11
But actual output should be:

15
Error exists
Model can fix in two ways:
Option 1 — Increase Weight

w = 2.8
New output:

5 × 2.8 + 1 = 15

Option 2 — Increase Bias

bias = 5
New output:

5 × 2 + 5 = 15

Option 3 — Adjust Both (Most Common)

w = 2.5
bias = 2.5
Output:

5 × 2.5 + 2.5 = 15
This is what happens in real models.

How Model Decides What to Adjust
Using:
- Backpropagation
- Gradient Descent

Model calculates:
- How much to change weight
- How much to change bias

Then updates both.

Simple Summary
During training:

Adjust weights
Adjust bias
Reduce error
Repeat

One Line Answer
Both weights and bias are adjusted during training to reduce error.
what is the input node how it comes from
Good question
Let's clearly understand **what an input node is and where it comes from**.
What is an Input Node?
In a Neural Network, an **input node** is simply **one piece of data (one feature)**.
Think of **input nodes = values you give to the model**.

Simple Example
Suppose you want to predict **Pass or Fail** based on:
- Study hours
- Sleep hours

Then:

Study hours → Input Node 1
Sleep hours → Input Node 2
Diagram:

Input Layer

(Study Hours) (Sleep Hours)
● ●
So:
- 2 values → 2 input nodes

Where Do Input Nodes Come From?
Input nodes come from **your dataset (features)**.
Example dataset:

Study	Sleep	Result
5	7	Pass
2	4	Fail

Here:
- Study → input node
- Sleep → input node

Example 2 — Face Detection
Input:
Image
But computer sees image as **pixels**:
Example:

120 130 140
125 135 145
Each number = **input node**
So:
- 100x100 image → 10,000 input nodes

This is used in
Convolutional Neural Network

Example 3 — Text (LLM)
Sentence:

"I love AI"
Converted to numbers:

[101, 305, 999]
Each number = input node
Used in
Large Language Model

General Rule
Number of input nodes = number of features
Example:

Features	**Input Nodes**
2 features	2 nodes
5 features	5 nodes
100 pixels	100 nodes

Visual Diagram

Input Layer

x1 ●
x2 ●
x3 ●
x4 ●

↓ Hidden Layer
Each **x1, x2, x3** is an input node.

One Line Answer
Input nodes come from your data — each feature/value becomes one input node.

Vector Database and Embedding

16 April 2026 19:23

Embedding - convert the words to numerical format
Vector Database - store the embedded datas.

Example - Apple fruits,Apple company

Sematic search - used to search based on context use the concept know as embedding

Apple is a fruit.
Apple is a commany

Is a fruit - [1,0]
Is a company - [0,1]

By checking the embedding they find the similarities

Various techniques can be used for word embedding called as **BERT,GPT,Word2Vec** etc

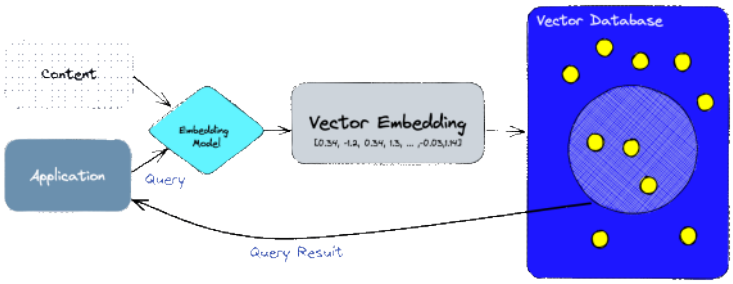
Store these in SQL Database

- In search query need to generate the embedding for it and use **cosine similarity** to check with the database
- But in considering the millions of records considering the cosine similarity usage of linear search which is taking more time
- For this we can use the concepts called **Indexing**
- **Hashing Function -** create buckets where they check the similarity and linear search done only within that bucket alone (called locality sensible hashing)

BENEFIT

- Faster search
- Optimised storage

A vector database indexes and stores vector embeddings for fast retrieval and similarity search, with capabilities like CRUD operations, metadata filtering, horizontal scaling, and serverless.



Feature	Vector Index (e.g., FAISS)	Vector Database (e.g., Pinecone)
Purpose	Focused on fast similarity search (ANN – Approximate Nearest Neighbor)	Full system to store, manage, and query vector embeddings
Data Management	No built-in support (you manage storage separately)	Supports insert, update, delete operations like a database
Metadata Handling	Limited or not supported	Supports metadata storage and filtering for advanced queries
Scalability	Requires manual setup (e.g., clustering, Kubernetes)	Built for automatic scaling and distributed processing
Real-time Updates	Often requires re-indexing to update data	Supports real-time or near real-time updates
Persistence / Storage	Usually in-memory or needs external storage integration	Persistent storage built-in
Backups & Recovery	Not available by default	Built-in backup and recovery (e.g., collections)
Ecosystem Integration	Requires custom integration	Easy integration with tools like ETL, analytics, and AI frameworks
Security &	Minimal or none	Built-in security, authentication

Lack in Traditional Database

Vector databases like Pinecone fulfill this requirement by offering optimized storage and querying capabilities for embeddings.

Traditional database lack the access of vector embedding.

Vector database was introduced to handle this kind of data by providing flexibility.

1. First, we use the embedding model to create vector embeddings for the content we want to index.
2. The vector embedding is inserted into the vector database, with some reference to the original content the embedding was created from.
3. When the application issues a query, we use the same embedding model to create embeddings for the query and use those embeddings to query the database for *similar* vector embeddings.

Traditional database store strings,numbers and other type of scalar data wherease the vector database store as the vectors
Traditional - Rows
Vector - Use similarity metrics
Uses different algorithm such as ANN (Approximate Nearest Neighbor) search.
Optimisation through the hashing,quantization or graph based search

Horizontal scaling = adding more machines (servers/nodes) to handle more data or traffic
Instead of making one machine stronger, you increase the number of machines.

Simple Example

Before scaling (single server)

- 1 server stores all vector embeddings
- Slow when data grows (e.g., millions of vectors)

After horizontal scaling

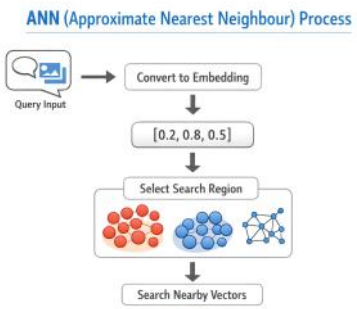
- Data is split across multiple servers

Server 1 → vectors 1–1M
Server 2 → vectors 1M–2M
Server 3 → vectors 2M–3M

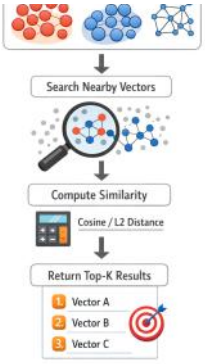
Queries are distributed
Faster retrieval
Can handle large-scale data

How it works in Vector Databases
In vector DBs (like Pinecone, Weaviate, etc.):

- Embeddings are **sharded** (split into parts)
- Each node stores a portion
- Query is sent to multiple nodes
- Results are combined



Ecosystem Integration	Requires custom integration	Easy integration with tools like ETL, analytics, and AI frameworks
Security & Access Control	Minimal or none	Built-in security, authentication, and access control
Multitenancy	Not supported	Supports namespaces / isolated partitions
Deployment Effort	High (manual setup required)	Low (managed or serverless options available)
Use Case	Best for research, prototypes, or local search systems	Best for production-grade AI applications (RAG, search, recommendations)



VECTOR INDEX - Store only the vectors not the metadata
VECTOR DATABASE - Include the vector index

What FAISS stores
- Vector embeddings (numeric arrays)
- Index structure for fast search

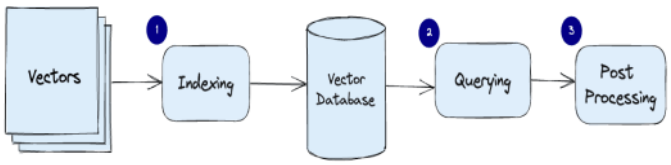
What FAISS does NOT store
- Metadata (e.g., name, category, text)
- Relationships between data
- Transactions (ACID)
- User access control
- Query filters (like SQL WHERE)

Common ANN Algorithms
- HNSW (graph-based)
- IVF (cluster-based)
- PQ (compressed vectors)

STEPS
- Dimensonality Reduction
- Metric Space (calculate distance)
- Indexing Structure (searching)

TYPES OF INDEXING

KD TREE
HNSW
LSH
ANALOGY



Indexing: The vector database indexes vectors using an algorithm such as PQ, LSH, or HNSW (more on these below). This step maps the vectors to a data structure that will enable faster searching.

Querying: The vector database compares the indexed query vector to the indexed vectors in the dataset to find the nearest neighbors (applying a similarity metric used by that index)

Post Processing: In some cases, the vector database retrieves the final nearest neighbors from the dataset and post-processes them to return the final results. This step can include re-ranking the nearest neighbors using a different similarity measure.

GENERATION DATABASE

The first generation is the vector database but speaking of AI evaluation led to develop more or go through the severless database.

Problem in first generation database
Seperation of storage from computer
Multitency
Freshness

Separation of storage from the computer

Coupling - how much two components depend on each other
Tight Coupling - two components are strongly dependent on each other
If one changes it will be reflected on the other function also

Example - Earphone and Phone If one broken the oterh will also be affected

Disadvantages
Hard to modify
Hard to rest
Low flexibility
Not reusable

Loose Cooupling

Components are independent that is intracting via interface and absrtaction

Example USB port can be plucked in Keyboard,Mouse and charger donot care about what device they are plugged in

Feature	Tight Coupling	Loose Coupling
Dependency	High	Low
Flexibility	Low	High
Code Change Impact	Large	Minimal
Reusability	Poor	Good
Testing	Difficult	Easy
Example	Direct object creation	Interfaces / DI

Tight Couling found in old vector system

Always running
Storage and compute together
Loose Coupling
Strogae seperated from compute
Runs only when needed
More scalable

Multitency

One system serving multiple users
Clinet use same application/database but their data is seperated
Example Gmail
Millons of users use the system
Email you can see many email
Others cannot access your data
Differnet login will be provided

In vector database
Data is divided using namespace
Each tenant gets its own logical partition
Queries only search within the tenants data

Advantages

- **Security** – data isolation
- ☑ **Cost saving** – one system for many users
- ☑ **Scalability** – easy to add more users
- ☑ **Efficiency** – shared infrastructure

Aspect	First-Gen Vector DB	Serverless Vector DB
Namespace handling	Logical only	Logical + resource-aware
Inactive tenants	Still consume resources	No cost
Compute usage	Always ON	On-demand
Cost efficiency	Poor	Optimized

Problem is everything is in the same index
So when system loads data it loads everything including inactive data so wastage of time and data
No true isolation at compute level
Even if:
- Tenant A is active
- Tenant B is inactive

👉 System still:
- Keeps B's data in memory
- Allocates resources for it

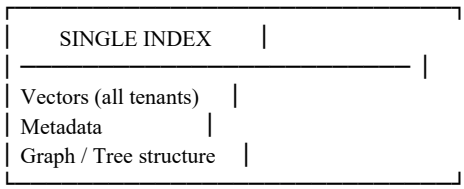
Real-Life Example
Imagine a **hotel**:
- 100 rooms (tenants)
- Only 10 guests staying

First-gen model:
- AC, lights, services ON for all 100 rooms
  👉 Waste of electricity (cost)

So overall problem is
Unnecessary cost
Reduced performance
Poor scalabilty
Inefficent storage used

Storage in First-Generation Vector DBs
🧠 **Core Idea**
👉 **Storage + Index + Compute are tightly coupled**

◇ **How data is stored**



🔍 **What this means**

- All vectors are stored in:
  - One **big index structure**
- Index itself contains:
  - Data + search structure (like graph/tree)

👉 Storage is **inside the index**

❌ **Problems**

1. No separation

- Cannot store data separately from compute
- Index must be:
  - Loaded into memory
  - Always available

2. Scaling issue

- More data → bigger index
- Bigger index → more RAM + compute

3. Multitenancy issue

- All tenants inside same index
- Even inactive data:
  - Stored + partially loaded
  - Affects cost

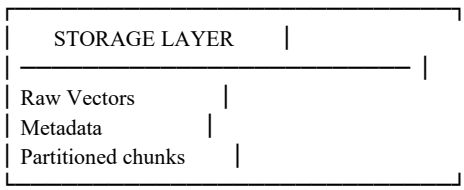
🟤 **2. Storage in Serverless Vector DBs**

(Used by systems like Pinecone)

🧠 **Core Idea**

👉 **Storage is separated from index + compute**

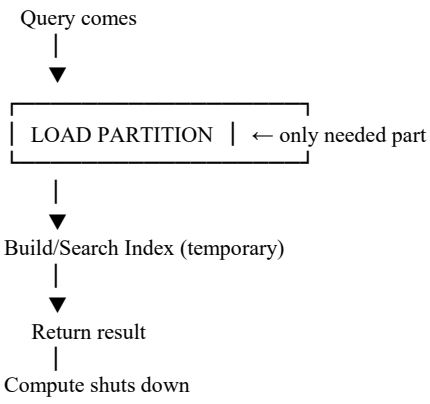
◇ **How data is stored**



👉 Stored in:

- Object storage (like blobs/files)
- Cheap and scalable

◇ **Index is NOT always active**



🔍 **What this means**

Storage Layer:

- Stores:

    - Vectors
    - Metadata
- Organized as:
    - **Partitions (chunks of similar data)**

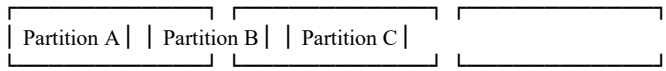
Index Layer:
- Created or loaded **on demand**
- Only for:
    - Required partitions

Compute Layer:
- Runs only during query
- Not always active

How Partitioned Storage Works
Instead of:

One Big File (All vectors)
We have:

Storage:



Each partition:
- Contains:
    - Similar vectors
    - Their metadata

Query Flow (Storage Perspective)
1. Query → converted to vector
2. System finds:
    - Relevant partition(s)
3. Only those partitions:
    - Loaded from storage
4. Temporary index created
5. Search performed
6. Compute stops

Side-by-Side Comparison

Feature	First-Gen Storage	Serverless Storage
Storage location	Inside index	Separate storage layer
Index	Always active	On-demand
Data structure	One large structure	Partitioned chunks
Loading	Full index loaded	Partial loading
Cost	High	Optimized
Scalability	Limited	High

Real-Life Analogy
First-Gen
- One huge book
- To read one topic:
    - Open entire book

Serverless
- Many small files (folders)
- Need one topic:
    - Open only that file

Final Core Insight
First-gen = data lives inside index
Serverless = data lives in storage, index is temporary

One-Line Summary
Serverless storage = partitioned data stored cheaply, indexed only when needed
Analogy (Best Way to Understand)
First-gen:
- One big hall with 100 people
- AC runs for entire hall ❌

Serverless:
- Smart building:
    - Rooms created dynamically
    - AC turns ON only where people are present

👉 Feels like each person has their own room

Feature	First-Gen	Serverless
Index structure	One big index	Partitioned / dynamic
Tenant isolation	Logical only	Logical + compute-level
Separate index per tenant	❌ No	⚠ Sometimes / effectively yes
Cost for inactive tenants	❌ Yes	☑ No

FIRST-GEN SERVERLESS
--- ---
One Big Index Multiple Partitions

[A B C D E] [A] [B] [C] [D]

Query A → searches almost everything ❌ Query A → goes ONLY to A partition ☑

Compute always ON ❌ Compute ON only when needed ☑

First-Gen Problem

- One index
- Shared compute
- Inactive tenants still cost money

☑ **Serverless Fix (like Pinecone)**

- Data split into partitions
- Query goes only where needed
- Compute runs **only for that tenant**

Step-by-step flow:

1. Data is stored in **multiple small index partitions**
2. Query comes
3. System decides:
   - Which partition(s) to search
4. Only those parts are:
   - Loaded
   - Searched
5. Compute shuts down after use

An index is partitioned based on **where vectors lie in the vector space (similarity / distance)**.
Let's make it very clear 👇

🔹 **Main Basis of Partitioning**

👉 **Vectors are grouped based on similarity (distance in space)**

- Similar vectors → same partition
- Dissimilar vectors → different partitions

🧠 **What "similarity" means**

In vector databases:

- Each item = a vector (numbers)
- Distance is measured using:
  - Cosine similarity
  - Euclidean distance

👉 Closer = more similar

🔬 **How Partitioning is Actually Done**

☑ **Method 1: Clustering (Most Common)**

Example using clustering:
You have vectors:

Dog → (0.1, 0.2)
Cat → (0.2, 0.1)
Car → (9.5, 9.8)
Bike → (9.2, 9.7)

Step 1: Find groups

Cluster 1 → Animals (Dog, Cat)
Cluster 2 → Vehicles (Car, Bike)

Step 2: Store separately

Partition A → Animal vectors
Partition B → Vehicle vectors

👉 This grouping is usually done using algorithms like:

- K-means clustering
- IVF (Inverted File Index) in FAISS

🔸 **Method 2: Centroid-Based Partitioning**
Each partition has a **center (centroid)**

Partition A center → (0.15, 0.15)
Partition B center → (9.3, 9.7)

When new vector comes:
👉 Assign it to nearest centroid

🔸 **Method 3: Graph-Based (Advanced)**
Used in HNSW-like systems:
- Vectors connected as neighbors
- Partitions formed based on connectivity

👉 Still based on **closeness**

🔍 **Query-Time Partition Selection**
When query comes:
1. Convert query → vector
2. Compare with partition centers
3. Pick nearest partitions
4. Search only those

Example
Query: “pet animal”

Query vector → close to Animal cluster
👉 Search:
- Partition A ☑
- Skip Partition B ❌

🔥 **Important Insight**
👉 Partitioning is **NOT based on:**
- IDs ❌
- timestamps ❌
- random split ❌

👉 It is based on:
- **semantic similarity (vector closeness)** ☑

🏢 **Real-Life Analogy**
Library 🏛️:
- Books grouped by topic:
  - Science
  - History
  - Sports

👉 You search:
- “Physics” → go to Science section only

🔳 **Final Flow**

Vectors → Embedded → Clustered → Stored in partitions

Query → Embedded → Find nearest cluster → Search there

🔸 **One-Line Answer**
👉 **An index is partitioned based on vector similarity (distance in embedding space)**

1. Problem Recap (Why this is needed)
☑ **Partitioning helps:**
- Search only a few partitions → fast + cheap

❌ **But creates a problem:**
👉 **Adding new data becomes slow**
Why?
- New vectors must be:
  - Clustered
  - Assigned to correct partition
  - Indexed properly

👉 This takes time ⏳

⚫ **Problem = Freshness Delay**

Insert → wait for partitioning → then searchable
👉 User inserts data now
👉 Cannot search immediately ❌

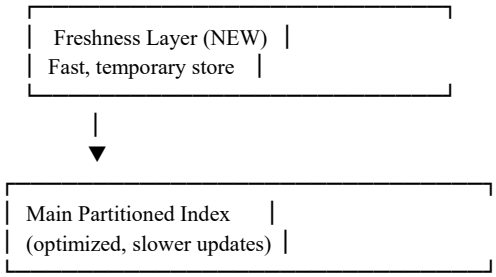
2. Solution: Freshness Layer (Important Concept)

Add a **temporary layer** before the main index

Idea

"Don't wait for full indexing — store new data somewhere quickly and make it searchable"

Architecture Overview



3. What Happens During Insert

Step-by-step:

User inserts vector
↓
Stored in Freshness Layer (immediate)
↓
Background process starts
↓
Vector is:
→ clustered
→ assigned to partition
→ added to main index

4. What Happens During Query

System searches **both layers**

Query
├──► Search Freshness Layer (new data)
├──► Search Main Index (old data)
↓
Merge results → Return

Key Insight

Freshness layer ensures:

- New data is **immediately searchable**
- While main index is **being updated in background**

Real-Life Analogy

Library

Main Index:

- Properly arranged books on shelves
- Fast to search
- But slow to update

Freshness Layer:

- New books placed on a **temporary table near entrance**

When searching:

- Check:
  - Table (new books)
  - Shelves (old books)

Why Not Directly Update Main Index?

Because:

Geometric Partitioning is Expensive

- Requires:
  - Finding correct cluster
  - Maintaining structure
  - Rebalancing partitions

👉 Doing this instantly = slow system

◇ 5. Trade-Off Solved

Problem	Solution
Slow indexing	Background processing
Poor freshness	Freshness layer
High latency	Partitioned index
Cost issues	Serverless compute

🔬 6. Internal Behavior

Freshness Layer:

- Small
- Fast lookup
- Often in-memory or fast storage

Main Index:

- Large
- Optimized for search
- Partitioned (clusters)

◇ 7. Final Combined Flow

INSERT:
User → Freshness Layer → Background → Main Index

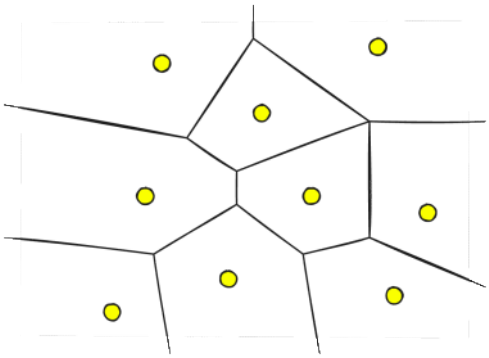
QUERY:
User → Freshness Layer + Main Index → Merge → Result

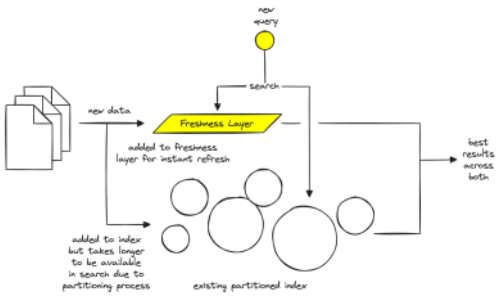
◇ Final Understanding

👉 **Freshness layer = temporary fast-access layer for new data**
👉 **Main index = optimized but slower-to-update structure**

◇ One-Line Summary

👉 **Freshness layer ensures instant searchability while the main partitioned index is being updated in the background**





Insert → Freshness Layer (temporary)
|
▼
Background Indexing
|
▼
Moved to Main Index

|
▼
Removed from Freshness Layer

Problem	Solution
Slow search	Partitioned index
Freshness delay	Freshness layer
High cost	Serverless compute
Multitenancy inefficiency	Usage-based grouping

Different algorithm are used for handling vector embeddings.

1).Random Projection
Create a matrix of random numbers
Create random projection matrix
What is Random Projection (Core Idea)
👉 **Random Projection = reducing high-dimensional vectors into lower dimensions using a random matrix while preserving similarity**

🔹 **2. Why Do We Need It?**
In AI / vector DBs:
- Embeddings can be:
  - 768D, 1024D, even 4096D 😵

❌ **Problems:**
- Slow search
- High memory usage
- Expensive computation

☑ **Solution:**
👉 Reduce dimensions:
- 1000D → 100D
- Faster + cheaper + scalable

🔹 **3. Intuition (Most Important)**
👉 We don't care about exact values
👉 We care about **distance between vectors**

Key Insight:
If two vectors are:
- Close before → remain close after
- Far before → remain far after

This is guaranteed (approximately) by
👉 Johnson–Lindenstrauss lemma

🔹 **4. How Random Projection Works (Step-by-Step)**

☑ **Step 1: Original Vector**
Example:

X = [1, 2, 3, 4] (4D)

☑ **Step 2: Create Random Matrix**
Suppose we want **4D → 2D**

```
R = [
  [0.5, -0.3],
  [0.2,  0.8],
  [-0.6, 0.1],
  [0.7, -0.5]
]
```
👉 Shape = (original_dim × new_dim)

☑ **Step 3: Multiply (Dot Product)**

X' = X × R
Result → smaller vector:

X' = [new_value1, new_value2]

🔹 **What This Does**
👉 Combines original features into fewer features
Think:
- Old → many features
- New → compressed features

5. Geometry Understanding

👉 Original vectors = points in high-dimensional space
👉 Projection = “shadow” in lower dimension

Analogy:

- 3D object → 2D shadow
- Shape still recognizable

6. Why “Random” Works

This is the surprising part 🦁
👉 Even random directions preserve distances approximately
Because:

- High-dimensional data has redundancy
- Random projection spreads information evenly

7. Mathematical Property

For two vectors **A and B**:
Before projection:

distance(A, B)
After projection:

distance(A', B') ≈ distance(A, B)
👉 Not exact, but **very close**

8. Types of Random Projection

1. Dense Random Projection

- Matrix filled with random numbers

2. Sparse Random Projection

- Mostly zeros
- Faster computation

3. Gaussian Projection

- Values from normal distribution

9. Where It Is Used

🔍 **In Vector Databases**
Used in systems like:

- FAISS

Purpose:

- Reduce vector size
- Speed up clustering
- Faster similarity search

10. Role in Full Pipeline

Raw Data
↓
Embeddings (high-dim)
↓
Random Projection (reduce size)
↓
Clustering (partitioning)
↓
Indexing
↓
Fast Search

11. Advantages

Advantage	Explanation
⚡ Fast	Simple matrix multiplication
💾 Memory efficient	Smaller vectors
📈 Scalable	Works for huge datasets
🧠 Simple	No complex training needed

12. Limitations

Limitation	Explanation
Approximate	Not exact distances
Randomness	Slight variation in results
Not always optimal	Other methods (PCA) may preserve more info

13. Random Projection vs PCA

Feature	Random Projection	PCA

Method	Random	Learned from data
Speed	Very fast	Slower
Accuracy	Approximate	Better
Complexity	Low	High

◇ 14. Full Intuition in One Shot

👉 Imagine:

- Huge detailed image (1000D)
- You compress it randomly

👉 Result:

- Smaller image
- Still looks similar
- Enough for recognition

◇ Final One-Line Summary

👉 **Random Projection = compress high-dimensional vectors using a random matrix while preserving relative distances**

Step 1: Take Two Similar Vectors
Imagine:

A = [1, 2, 3]
B = [2, 4, 6]

👉 Clearly:

- B is just A × 2
- So they are **very similar**

◇ Step 2: Create a Random Matrix

We reduce from **3D → 2D**

```
R = [
  [1,  0],
  [0,  1],
  [1, -1]
]
```

◇ Step 3: Multiply (Dot Product)

For A:

A = [1, 2, 3]

New A:

= (1×1 + 2×0 + 3×1 , 1×0 + 2×1 + 3×(-1))
= (1 + 0 + 3 , 0 + 2 - 3)
= (4, -1)

For B:

B = [2, 4, 6]

New B:

= (2×1 + 4×0 + 6×1 , 2×0 + 4×1 + 6×(-1))
= (2 + 0 + 6 , 0 + 4 - 6)
= (8, -2)

◇ Step 4: Compare Results

Before Projection:

A = [1,2,3]
B = [2,4,6] → very similar

After Projection:

A' = (4, -1)
B' = (8, -2)

Step 1: We Already Have Reduced Vectors
From previous example:

A' = (4, -1)
B' = (8, -2)
Let's add a few more:

C' = (1, 5)
D' = (2, 6)

◇ Step 2: Visualize Them (Very Important)

Think of them on a graph:

- A' (4, -1)
- B' (8, -2) → close to A'
- C' (1, 5)
- D' (2, 6) → close to C'

👉 So naturally we see **2 groups**

🔹 **Step 3: Group Based on Distance**

👉 We group vectors that are **close to each other**

Group 1:

A', B' (near each other)

Group 2:

C', D' (near each other)

🔹 **Step 4: Create Partitions**

Now each group becomes a **partition (sub-index)**

Partition 1 → A', B'
Partition 2 → C', D'

Product Quantisation

Compress a big vector into a small code while keeping similarity information.
Original vector - large
PQ - converts into compact codes

Name the codes

Real-Life Analogy

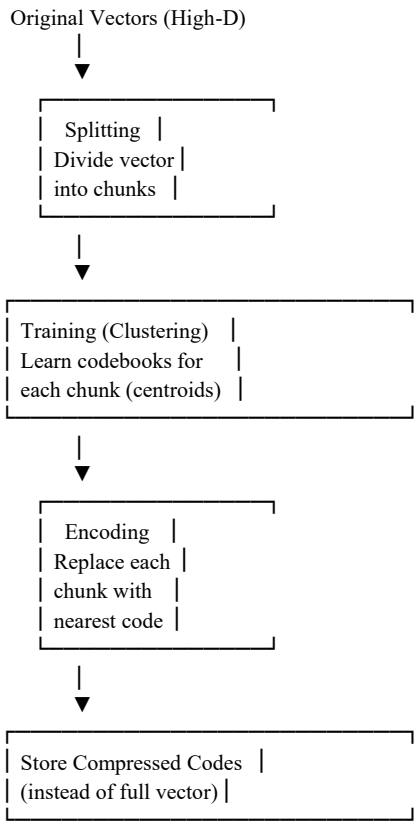
Imagine describing a person:
Instead of:

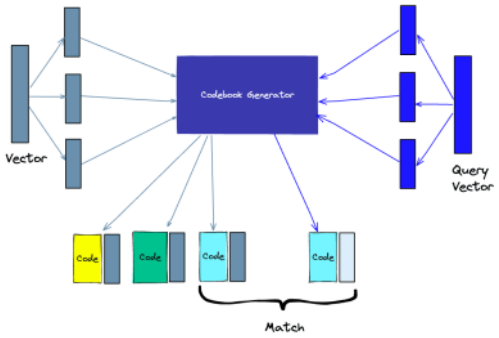
Height: 172.3 cm
Weight: 68.7 kg
Age: 23.4
You store:

Height → Medium
Weight → Fit
Age → Young

👉 You lose exact precision

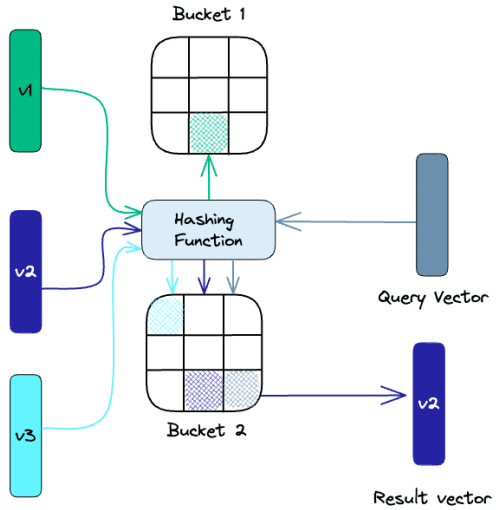
👉 But still enough for comparison ☑





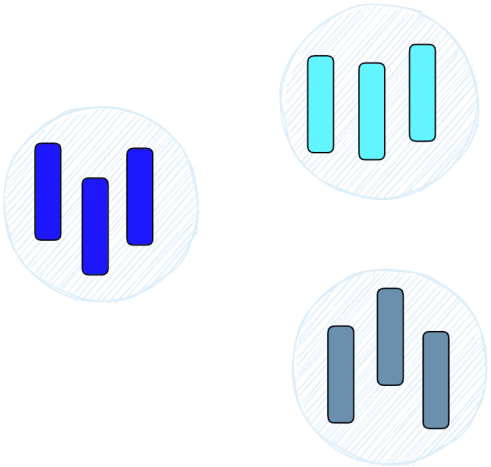
1. Splitting -The vectors are broken into segments.
2. Training - we build a “codebook” for each segment. Simply put - the algorithm generates a pool of potential “codes” that could be assigned to a vector. In practice - this “codebook” is made up of the center points of clusters created by performing k-means clustering on each of the vector’s segments. We would have the same number of values in the segment codebook as the value we use for the k-means clustering.
3. Encoding - The algorithm assigns a specific code to each segment. In practice, we find the nearest value in the codebook to each vector segment after the training is complete. Our PQ code for the segment will be the identifier for the corresponding value in the codebook. We could use as many PQ codes as we’d like, meaning we can pick multiple values from the codebook to represent each segment.
4. Querying - When we query, the algorithm breaks down the vectors into sub-vectors and quantizes them using the same codebook. Then, it uses the indexed codes to find the nearest vectors to the query vector.

Locality - sensitve hashing



Hierarchical Navigable Small World (HNSW)

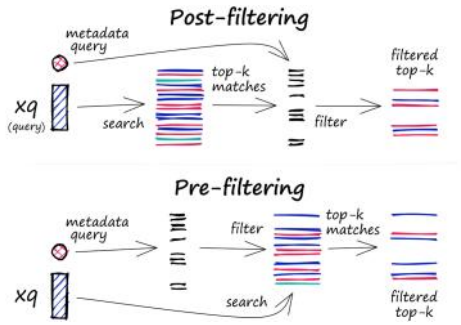
HNSW creates a hierarchical, tree-like structure where each node of the tree represents a set of vectors. The edges between the nodes represent the similarity between the vectors. The algorithm starts by creating a set of nodes, each with a small number of vectors. This could be done randomly or by clustering the vectors with algorithms like k-means, where each cluster becomes a node.



Similarity Measures

- Cosine similarity: measures the cosine of the angle between two vectors in a vector space. It ranges from -1 to 1, where 1 represents identical vectors, 0 represents orthogonal vectors, and -1 represents vectors that are diametrically opposed.
- Euclidean distance: measures the straight-line distance between two vectors in a vector space. It ranges from 0 to infinity, where 0 represents identical vectors, and larger values represent increasingly dissimilar vectors.
- Dot product: measures the product of the magnitudes of two vectors and the cosine of the angle between them. It ranges from -∞ to ∞, where a positive value represents vectors that point in the same direction, 0 represents orthogonal vectors, and a negative value represents vectors that point in opposite directions.

Filtering

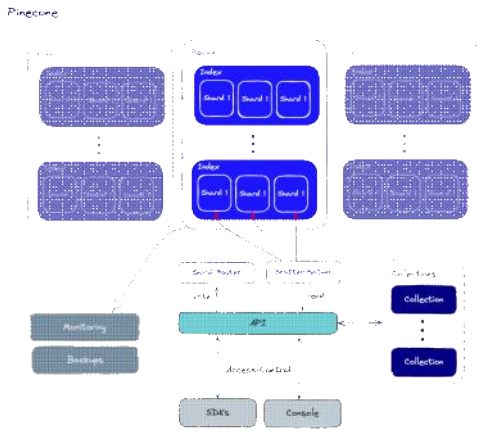


The filtering process can be performed either before or after the vector search itself, but each approach has its own challenges that may impact the query performance:

- Pre-filtering: In this approach, metadata filtering is done before the vector search. While this can help reduce the search space, it may also cause the system to overlook relevant results that don't match the metadata filter criteria. Additionally, extensive metadata filtering may slow down the query process due to the added computational overhead.
- Post-filtering: In this approach, the metadata filtering is done after the vector search. This can help ensure that all relevant results are considered, but it may also introduce additional overhead and slow down the query process as irrelevant results need to be filtered out after the search is complete.

Database Operations

Unlike vector indexes, vector databases are equipped with a set of capabilities that makes them better qualified to be used in high scale production settings. Let's take a look at an overall overview of the components that are involved in operating the database.



To ensure both high performance and fault tolerance, vector databases use sharding and replication apply the following:

1. Sharding - partitioning the data across multiple nodes. There are different methods for partitioning the data - for example, it can be partitioned by the similarity of different clusters of data so that similar vectors are stored in the same partition. When a query is made, it is sent to all the shards and the results are retrieved and combined. This is called the “scatter-gather” pattern.
2. Replication - creating multiple copies of the data across different nodes. This ensures that even if a particular node fails, other nodes will be able to replace it. There are two main consistency models: *eventual* consistency and *strong* consistency. Eventual consistency allows for temporary inconsistencies between different copies of the data which will improve availability and reduce latency but may result in conflicts and even data loss. On the other hand, strong consistency requires that all copies of the data are updated before a write operation is considered complete. This approach provides stronger consistency but may result in higher latency.

RAG

17 April 2026 00:45

RAG - means Retrieval Augmented Generation

It is used to
- improve accuracy,
- Relevance
- freshness by combining information retrieval and text generation

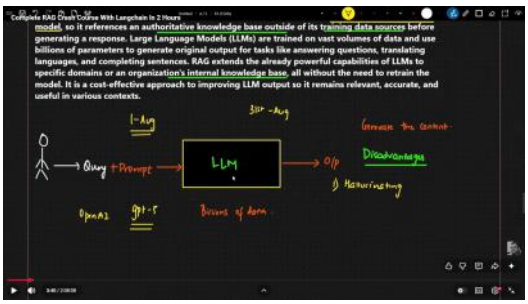
It is the process of optimizing the output of a LLM model so it references an authoritative knowledge base outside of its training data source before generating a response.

LLM are trained on vast volumes of data and use billions of parameters to generate original output for tasks like answering questions, translating language and completing sentence

It is cost effective approach to improve LLM output so it remains relevant, accurate and useful in various contexts.

Problems of LLM without RAG

1).Hallucination



- LLM trained only till dates within August 1
- If I needed data relevant to August 31 it result in Hallucination problem.
- The problem is generating information that sounds correct but is actually false, made up or unsupported.
- To Solve this problem RAG can be used

For A Particular Company

- I have a data such as policies, HR and finance for my particular purpose or for my own company.
- So I need to train the LLM with my data also. These data can also be updated often so for this RAG is often used.

Usage of RAG

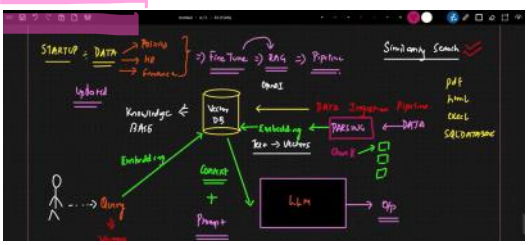
- Usage of vector database.
- There are two types of pipelines
  - Data Injestion Pipeline
  - Retrieval Pipeline

In data ingestion pipeline

DATA - PARSING - EMBEDDING

- The data includes pdf,html,excel and SQL database
- The data is converted in chunk commonly know as Parsing
- It is passed to the embedding process and store the data on the vector database
- EMBEDDING - Convert text to vector

There are various source for embedding model such as OpenAI gpt etc

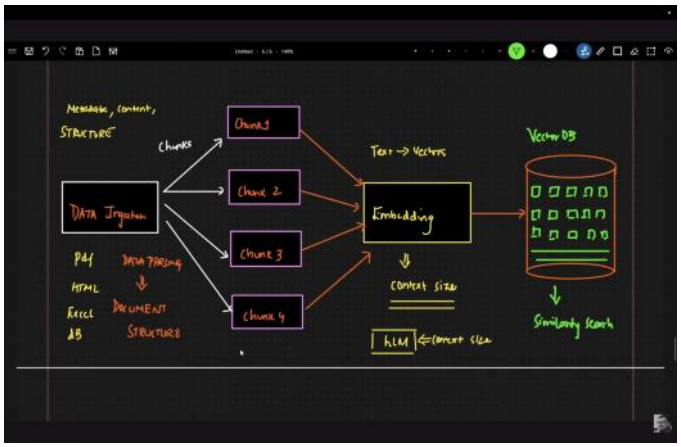


DATA INGESTION PIPELINE

How to perform the Data Parsing
For this needed to learn about the document structure

First Data Parsing - Chinking - Embedding

- Data parsing is the process of taking raw data and converting it into a structured, usable format.
- Chunking is the process of breaking large information into smaller, manageable pieces (“chunks”) so it is easier to process, understand, or store.
- Embedding - convert text to vectors and store in the vector database
- In vector DB similarity search is used to retrieve the data's.



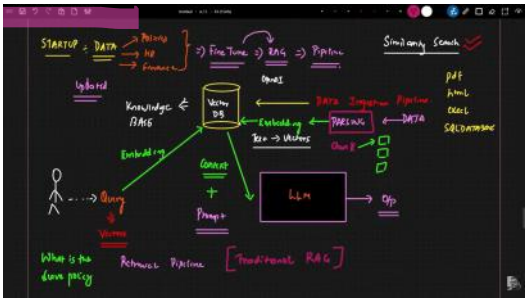
Langchain

Chatgpt is not LLM it uses the OpenAI API to call the LLM like GPT3.4 etc.
(Internally uses the LLM)

LLM - generate the human like text

OpenAI API is not enough to build an LLM
So to built a LLM application uses the LANGCHAIN
LANGCHAIN - It is a framework that allows to build application along with LLM
User to add our data
Usage of memory
Usage of tools/actions
LLM cannot directly use the datas given by us to give the ouput so we need to use LANGCHAIN

Usage of OpenAI API is cost so I prefer the OLLAMA



Here the problem is generally solved using the vector Database.
The query is embedded into vector DB all the updated data are stored in the vector db using only the data ingestion pipeline and the question asked by the user is passed into the same embedding model and similar data are collected using the similarity search the alogrithm that are found in the vector embedding model.
That content is passed into the LLM and the output is generated.
This is the traditional RAG

Doubt is
The data can be passed directly into llm
Here the data are passed as input to the LLM from the vector database
In RAG, data is not directly inserted into the LLM. Instead, relevant information is retrieved from a vector database and passed as context in the prompt at runtime. The LLM uses this context to generate accurate responses without being retrained.

PERPLEXITY - RAG application

Perplexity is used to measure how well a large language model is trained.

- **Low perplexity** → model is confident & accurate
- **High perplexity** → model is confused

Perplexity is based on probability of a sequence.
It is commonly defined as:

$PP(W) = P(w_1, w_2, \dots, w_n)^{-\frac{1}{n}}$

Where:

- W = sequence of words
- $P(W)$ = probability assigned by the model
- n = number of words

Perplexity is a metric that measures how well a language model predicts a sequence of words, with lower values indicating better performance.

Simplified form (most used):

$$\text{Perplexity} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P\left(w_i\right)\right)$$

Step-by-step calculation
Example sentence:
"I love AI"
Assume model probabilities:

- P(I) = 0.5
- P(love | I) = 0.4
- P(AI | I love) = 0.1

Step 1: Take log probabilities

$\log(0.5), \log(0.4), \log(0.1)$

≈

- -0.693
- -0.916
- -2.303

Step 2: Average them

$$\frac{-0.693 - 0.916 - 2.303}{3} = -1.304$$

Step 3: Multiply by -1

1.304

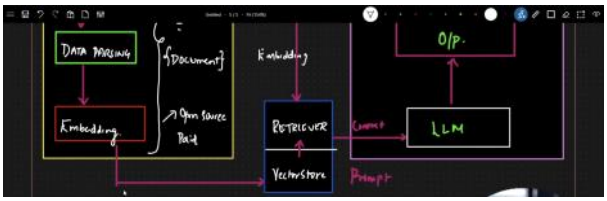
Step 4: Exponentiate

Perplexity = $e^{1.304} \approx 3.68$

Interpretation

- Perplexity ≈ 1 → perfect prediction
- Perplexity ≈ 10–50 → normal language model
- Lower is always better

RAG



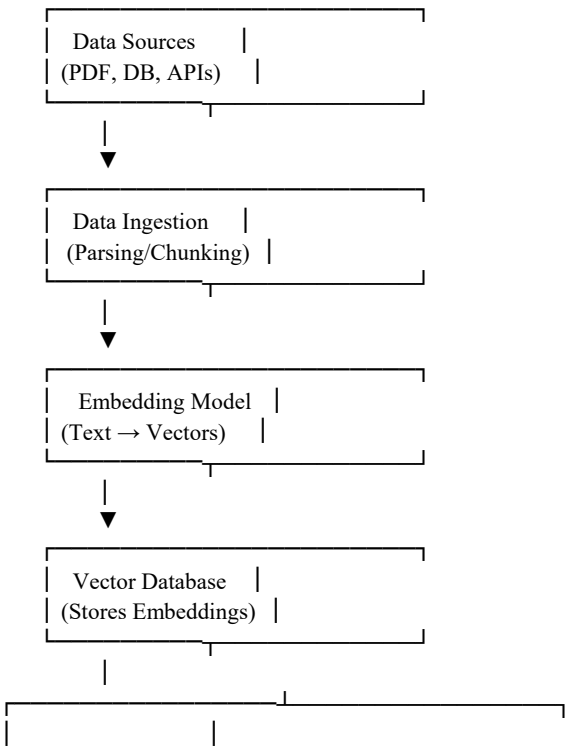
- Retrieve relevant data from external sources
- Augment the prompt with that data
- Generate a better answer using an LLM

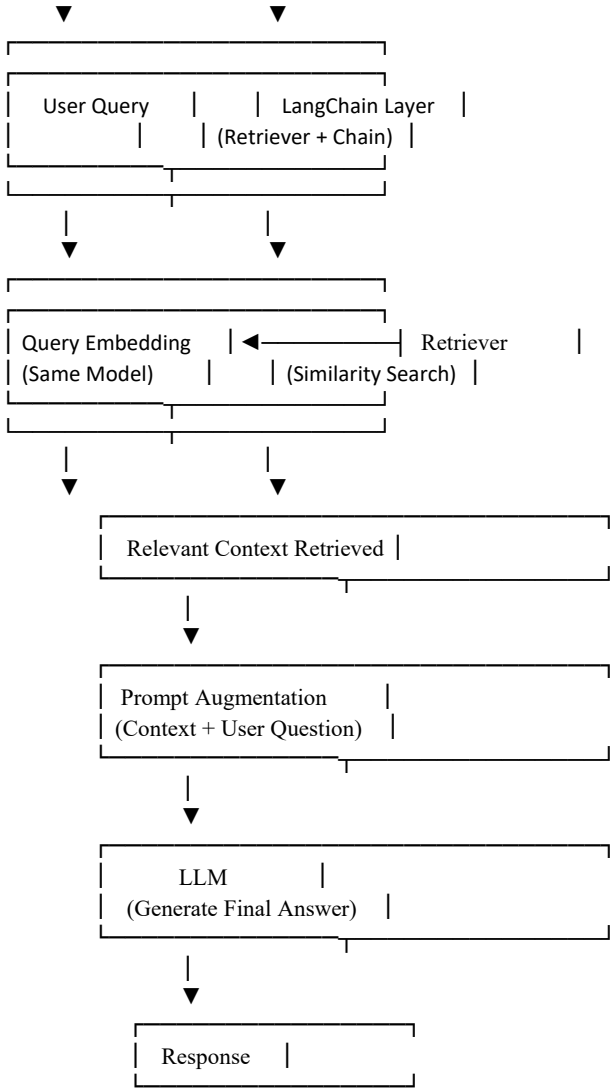
Question -> LLM -> Data Sources -> Return Answers

LangChain Document Structure

Generate text
Models
Prompts
Chains
Memory

User Question
→ Retriever (search vector DB)
→ Get relevant data
→ Combine with prompt
→ LLM generates answer





Data → Chunk → Embed → Store → Retrieve → Augment → Generate

Langchain makes builduing LLM much easier
It is a python library that makes builduing LLM apps much easier

from langchain

These are the loaders which load the content and give output in form of document structure

Main Loaders
PDFLoader
CSVLoader
WebBaseLoader
DirectoryLoader

```
from langchain.document_loaders import PyPDFLoader;
loader = PyPDFLoader('file.pdf');
document = loader.load();
```

Output will be in document structure

```
from langchain_core.documents import Document;

Doc = Document(
Page_content = "this is the main content",
Meta_data = {
"source":"example.txt",
"page":1,
"auhtor":"Siva Kavindra",
```

"date_created":"2025-01-01"
})

Core Components of the Langchain Document

Page_content (str)
Metadata (dict)

Page Content
Actual text content
Metadata is additional details that are
Number of pages
Timestamp etc

""" """ (triple quotes) are used for **multi-line strings** or **docstrings**.
1. Multi-line string
Yes — **data stays exactly as it is** (including spaces, new lines).
Example:

```
text = """Hello
This is line 2
This is line 3"""

print(text)
```

Creating a RAG Project

18 April 2026 13:39

UV Package
uv is a fast Python package manager (alternative to pip/venv), developed by Astral.

uv init
↓
pyproject.toml created
↓
uv venv
↓
.venv (virtual environment)
↓
uv add <package>
↓
Dependencies installed + uv.lock
↓
uv sync
↓
Reproducible environment
↓
uv run app.py

File	Purpose
pyproject.toml	Project config + dependencies
uv.lock	Exact versions (like lock file)
.venv/	Virtual environment

Feature	pip	poetry	uv
Speed	Slow	Medium	⚡ Fast
Dependency mgmt	❌	☑	☑
Virtual env	Manual	Auto	Auto
Lock file	❌	☑	☑

2nd uv venv
Creates a virtual environment
Activate the virtul environment
Create requirements.txt file and add needed packages that needed to be installed
Include all the needed package that are needed for the project

As all the chunks are stored in the form of document structure

Document loader will have the many way including cloud extraction also

Once the data are converted into document structure next is chucking and next id emedding and next is vector database

Reasoning model -
It is a process of answering question that require complex, multi-step generation with intermediate steps.

What is the capital of India ? Non reasoning
Why do you think …? Or any calculation - reasoning

Generic LLM - Plain response
Reasoning LLM
- Not visible to users
- Visible to user

When to use Reasoning problem

Complex tasks such as solving puzzles, advanced math problem and challenging coding tasks

For simple tasks like summarization, translation and knowledge based sharing - no needed reasoning model because cost will be higher

Reasoning
- Step by step problem solving
- Breakdown problem into smaller sub steps and shows intermediate reasoning ("thinking phase")
- Highly structural
- Easy to detect the errors
- Example - openAI,DeepSeek,o1-mini,o3-mini
- Use cases - reasoning, legal analysis, complex problem solving, AI agents with multi step decision making

Generic LLM
- Text generation and understanding
- Show output without showing any steps
- More flexible
- Black box nature makes reasoning for output much harder
- Example - GPT, Llama, Claude
- Use case - chatbots, text summarisation, content generation, Q&A, code completion

	Generic Model	Reasoning Model
Example	Claude Sonnet 4.6	OpenAI o1, o3
Approach	Responds directly	Thinks step-by-step internally before answering
Speed	Faster	Slower
Best for	Everyday tasks, writing, coding, chat	Complex math, logic puzzles

Real-World Example
Task 1: Write Email
Use → **Generic LLM**
Task 2: Solve Complex SQL Optimization
Use → **Reasoning LLM**
Task 3: Build AI Agent
Use → **Reasoning LLM**
Task 4: Summarize Document
Use → **Generic LLM**

What is a Reasoning LLM?
A **Reasoning LLM** is designed to **think step-by-step** before answering.
It **breaks down complex problems** into smaller parts and solves them logically.

Key Characteristics
- Step-by-step thinking
- Multi-step problem solving
- Logical decision making
- Better accuracy for complex tasks
- Slower but more reliable

Example
Question:
If a train travels 60 km/h for 2.5 hours, how far does it go?
A **Reasoning LLM** would think like:
Step 1 → Formula = Distance = Speed × Time
Step 2 → Distance = 60 × 2.5
Step 3 → Distance = 150 km
Final Answer → **150 km**
This is **step-by-step reasoning**.
Popular Reasoning Models
- OpenAI **o1 / o3 models**
- OpenAI **o1-mini**
- OpenAI **o3-mini**
- DeepSeek **DeepSeek-R1**
- Google DeepMind **Gemini reasoning models**

These models **internally reason** before giving output.

What is Generic LLM?
A **Generic LLM** is designed for **text generation and understanding**, not deep reasoning.
It **directly gives the answer** without step-by-step thinking.
Language generation means it gives the output as the content if I am asking

Key Characteristics
- Fast responses
- Flexible conversation
- Good for general tasks
- Less reliable for complex logic
- "Black box" behavior

Example
Question:
What is the capital of India?
Generic LLM Answer:
New Delhi
No reasoning needed.
Direct answer.

Popular Generic LLMs
- OpenAI **GPT-4 / GPT-4o**
- Meta **Llama**
- Anthropic **Claude**
- Google **Gemini Flash**

Modern Trend (Very Important)
Most new models today are **Hybrid models**
They can **switch between reasoning & generic modes automatically**.
Examples:
- OpenAI GPT-5-style models
- Anthropic Claude 3+
- Google Gemini 1.5+

These models:
- Use reasoning only when needed
- Use generic response when simple

This **reduces cost + improves speed**

How Generic LLM Answers Without Step-by-Step Thinking
Generic LLMs **don't actually "think" like humans**.
They **predict the most likely next word** based on patterns learned during training.
Example
Question:
What is the capital of India?
The model has seen **millions of examples** like:
- "Capital of India is New Delhi"
- "India → New Delhi"
- "New Delhi is the capital"

So it **predicts directly**:
→ **New Delhi**
No reasoning needed — just **pattern recognition**.

Example with Calculation
Question:
25 × 4
Generic LLM doesn't calculate step-by-step.
Instead, it has learned:
- "25 × 4 = 100" appears often
- So it **predicts → 100**

This is called:
Pattern-based answering

Reasoning Model (Different Behavior)
Reasoning model may internally do:
25 × 4
= 20 × 4 + 5 × 4
= 80 + 20
= 100
So it's **actually reasoning**.

FOR GENRIC

A **token** is a **small piece of text** that the

FOR GENRIC

User Input
↓
Tokenization
↓
Convert to Numbers (Embeddings)
↓
Transformer Model (LLM)
↓
Predict Next Word Probabilities
↓
Pick Best Word
↓
Repeat Until Complete

A **token** is a **small piece of text** that the model uses instead of full sentences.
Tokens can be:

- Full words
- Part of words
- Characters
- Spaces
- Punctuation

ChatGPT Versions

20 April 2026 23:35

AI Evolution — Complete Notes

1️⃣ GPT-1 (2018)

Model: GPT-1

Problem Before

- Rule-based chatbots
- No real language understanding

What GPT-1 Introduced

- Transformer architecture
- Predict next word

Problems Still

- Very small
- Poor reasoning
- Bad conversations

👉 **Result:** First LLM proof

2️⃣ GPT-2 (2019)

Model: GPT-2

Problem in GPT-1

- Small model
- Weak text generation

What GPT-2 Improved

- Larger model
- Better paragraphs
- More natural language

Problems Still

- Hallucinations
- No reasoning
- Not reliable

👉 **Result:** First impressive text generation

3️⃣ GPT-3 (2020)

Model: GPT-3

Problem in GPT-2

- Not useful for real tasks

What GPT-3 Improved

- Much larger model
- Coding ability
- Better understanding

🧠 IMPORTANT: RAG STARTED HERE (2020)

Technique Introduced:

Retrieval-Augmented Generation

Why RAG Was Needed

GPT-3 Problems:

- Hallucinations
- Outdated knowledge
- No private data access

RAG Solution

- Retrieve documents
- Add context
- Generate answer

👉 **Result:** First real-world AI

4️⃣ GPT-3.5 (2022)

Model: GPT-3.5

Problem in GPT-3

- Not conversational
- Hard to control

What GPT-3.5 Improved

- Better chat
- Instruction following
- Better coding

Major Event

ChatGPT launched

RAG Usage

- Chat with PDFs
- Enterprise search
- Document QA

👉 **Result:** ChatGPT becomes popular

5️⃣ GPT-4 (2023)

Model: GPT-4

Problem in GPT-3.5

- Weak reasoning
- Bad logic

What GPT-4 Improved

- Strong reasoning
- Better coding
- Better accuracy

RAG Became Very Popular

Used for:

- Company knowledge bases
- Chat with documents
- AI assistants

👉 **Result:** Professional AI

6️⃣ GPT-4o (2024)

Model: GPT-4o

Problem in GPT-4

- Slow
- Expensive

What GPT-4o Improved

- Faster

- Voice
- Image support
- Cheaper

RAG Evolution

- Real-time retrieval
- Multi-modal RAG (image + text)

👉 **Result:** Real-time AI

7️⃣ Reasoning Models Era

o1 (Reasoning Breakthrough)

Model: OpenAI o1

Problem Before

- Weak deep reasoning

What o1 Improved

- Step-by-step thinking
- Better math
- Better debugging

RAG Improvement

- Reasoning + RAG combined

👉 **Result:** Deep thinking AI

o3 (Better Reasoning)

Model: OpenAI o3

Problem in o1

- Slow reasoning

What o3 Improved

- Faster reasoning
- Better logic

👉 **Result:** Faster reasoning

8️⃣ GPT-5 (New Generation)

Model: GPT-5

Problem Before

- Separate models:
- Fast models
- Smart models

What GPT-5 Improved

- Combined speed + reasoning

RAG Improvement

- Native RAG pipelines
- Long context retrieval

👉 **Result:** Balanced AI

9️⃣ GPT-5.1

Model: GPT-5.1

Problem in GPT-5

- Some hallucinations

What GPT-5.1 Improved

- Better accuracy
- Better reliability

👉 **Result:** Stable GPT-5

🔟 GPT-5.2

Model: GPT-5.2

Problem in GPT-5.1

- Reasoning still improving

What GPT-5.2 Improved

- Better reasoning
- Better technical explanations

👉 **Result:** Smarter GPT-5

1️⃣1️⃣ GPT-5.3 (Current — Me)

Model: GPT-5.3

Problem in GPT-5.2

- Speed vs reasoning balance

What GPT-5.3 Improved

- Faster
- Smarter
- Better coding
- Better teaching

Modern RAG

- Agentic RAG
- Hybrid search
- Long context retrieval

👉 **Result:** Most advanced

📊 RAG Evolution Summary

Model	RAG Status
GPT-1	❌ No
GPT-2	❌ No
GPT-3	☑ Introduced
GPT-3.5	☑ Popular
GPT-4	☑ Widely used
GPT-4o	☑ Real-time
o1	☑ Reasoning + RAG
GPT-5	☑ Native RAG
GPT-5.3	☑ Advanced RAG